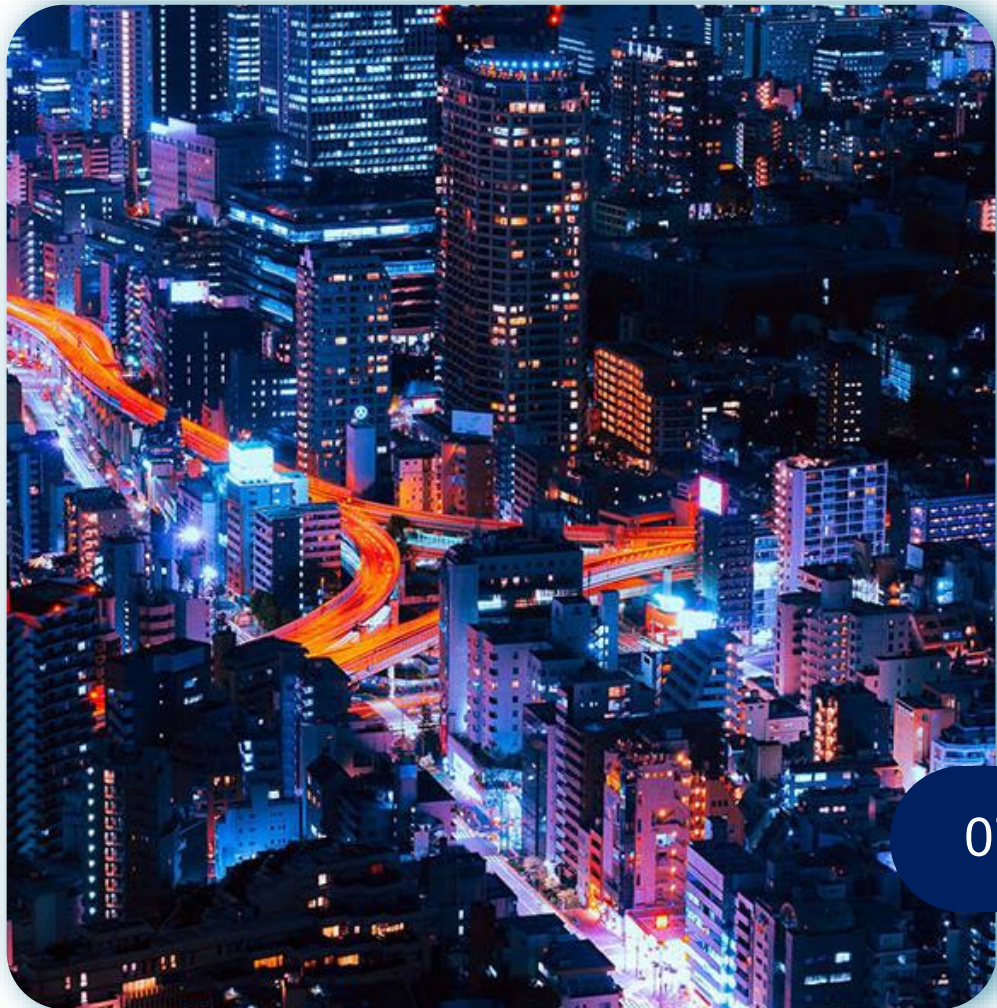




Apprentissage supervisé

DSA 2026

UTHAYASOORIYAR Benno / LY Antoine

07/09/2026

Objectif et structure de la session

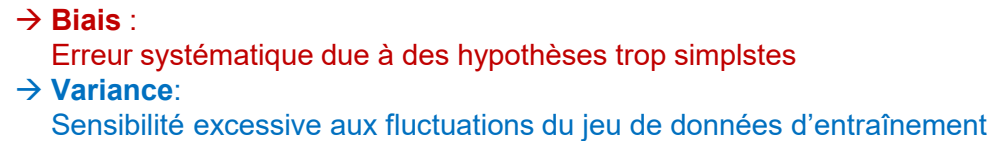
Comprendre comment les algorithmes apprennent, pourquoi certains sont meilleurs que d'autres, et comment choisir le bon outil pour chaque problème actuariel.

Bloc 1	Comment fonctionne le .fit() ?	1h45	<i>Stats et ML partagent un objectif mais diffèrent en philosophie</i>
Bloc 2	Les algorithmes d'ensemble supervisés	1h45	<i>Le cœur du ML = <u>généraliser</u>, pas fitter</i>

Pré-requis

- Biais-variance
- Cross-validation
- Pipelines sklearn
- Régularisation

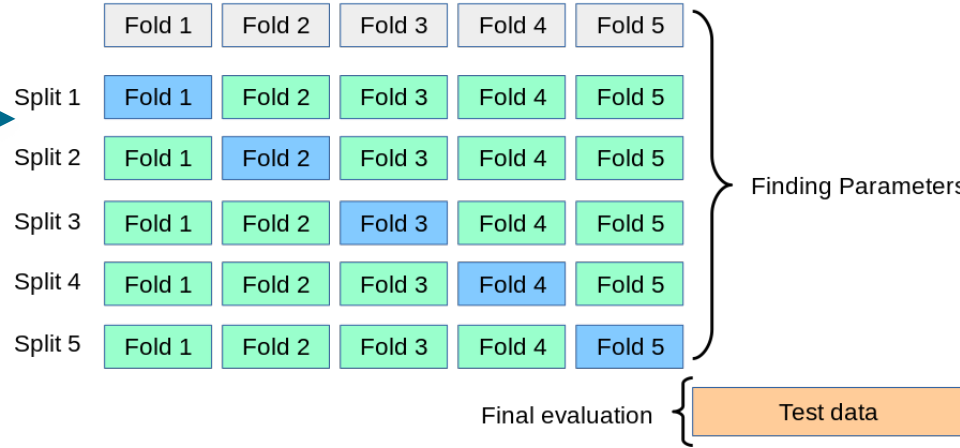
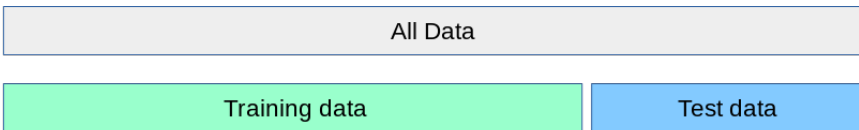
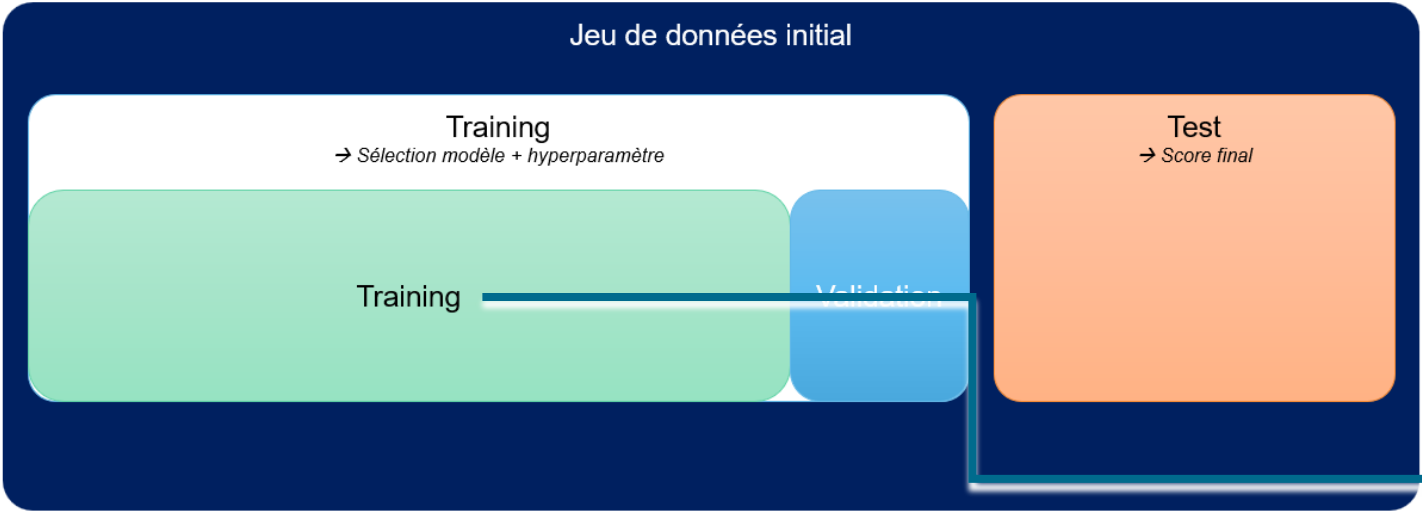
Analogie pédagogique



Protocole de validation

Validation croisée

Multiplier les partitions du training set, pour multiplier les scénarios d'entraînement



⚠ Attention : la validation croisée permet d'évaluer des approches différentes sur un même dataset en comparant une performance moyenne suite aux entrainements faits par chaque approche. Elle ne permet pas d'obtenir la meilleur combinaison de paramètres pour une approche donnée (= feature engineering).
C'est une méthode de validation et non d'optimisation des paramètres.
Ex: On souhaite comparer une approche de Boosting vs une régression Logistique sur de la probabilité de rachat sur un portefeuille d'assurance vie (0= pas de rachat, 1= rachat). Pour cela, la validation croisée va permettre de comparer équitablement les deux approches sur le même dataset.

Scikit-Learn

Les Pipelines

Un framework pour éviter les erreurs classiques

Le pipeline concentre les étapes de preprocessing et de modélisation dans le même objet Python. Des règles sont ensuite appliqués pour éviter les pièges habituels:

- Pas de data leakage
- Reproductibilité: tout le preprocessing + le modèle en un seul objet
- Déploiement facile (sauvegarde du modèle en pkl)
- Cross-validation propre : le preprocessing est refait à chaque fold

```
from sklearn.pipeline import Pipeline, make_pipeline

# Méthode 1 : Pipeline explicite (avec noms)
pipe = Pipeline(steps=[
    ('scaler', StandardScaler()),
    ('model', Ridge(alpha=1.0))
])

# Méthode 2 : make_pipeline (noms automatiques)
pipe = make_pipeline(StandardScaler(), Ridge(alpha=1.0))

# Utilisation : exactement comme un modèle simple !
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
score = pipe.score(X_test, y_test)
```

Bloc 1

Comment fonctionne le `.fit()` ?

Machine Learning

Ce qui se passe quand vous appelez .fit()

Chaque algorithme minimise le risque empirique, mais avec une stratégie différente

$$\hat{\mathcal{R}}_n(m) = \frac{1}{n} \sum_{i=1}^n l(m(x_i), y_i)$$

.fit() cherche les paramètres qui réduisent cette perte sur les données d'entraînement.

```
model.fit(X_train, y_train)
```



1 Vérifier les données — dimensions, types, valeurs manquantes

2 Initialiser les paramètres internes

3 **BOUCLE D'OPTIMISATION**

- Calculer la perte actuelle
- Chercher comment l'améliorer
- Mettre à jour les paramètres
- Vérifier la convergence

4 Stocker les paramètres appris — coef_, tree_, ...



```
model.predict(X_new) # utilise les paramètres appris
```

Même objectif, moteurs différents

Régression linéaire

solution analytique (pas d'itération)

Régression logistique

gradient · Newton-Raphson / L-BFGS

Arbre de décision

algorithme glouton · splits récursifs

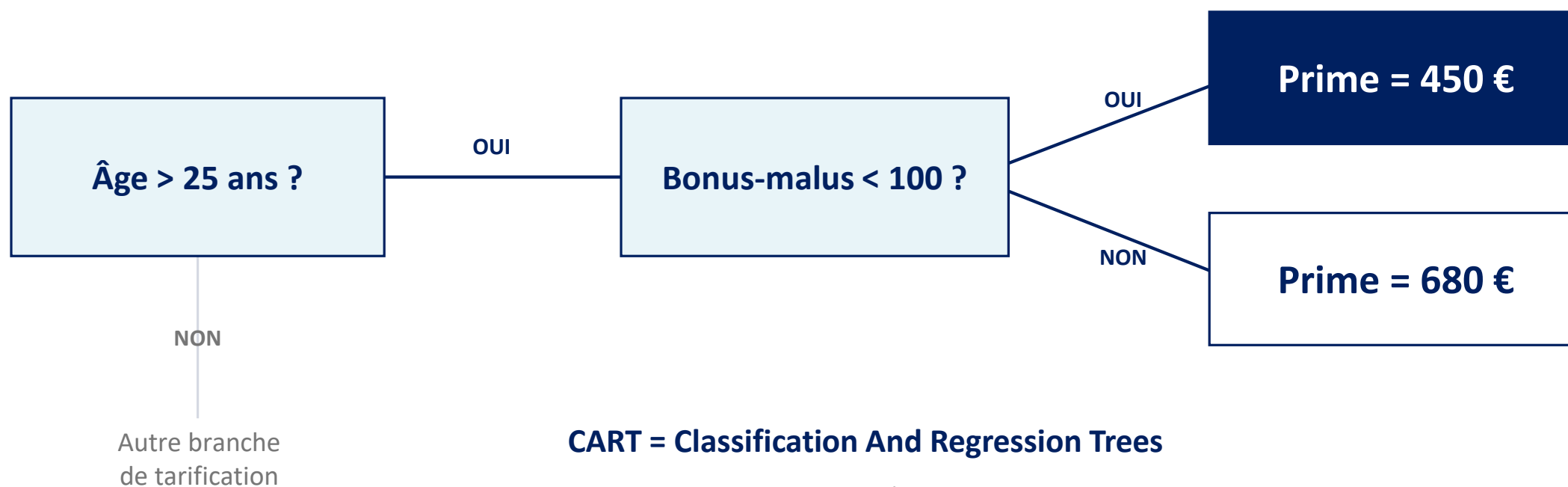
GBM / XGBoost

arbres séquentiels sur les résidus

Machine Learning

CART : un questionnaire à choix binaires

Une suite de questions oui / non transforme des profils en décisions explicables



CART = Classification And Regression Trees

Breiman et al., 1984

Intuitif à lire et interprétable

Machine Learning

CART — construction récursive

*À chaque nœud, l'algorithme se demande:
« quelle est la meilleure question à poser ICI, sur les données qui sont arrivées jusqu'ici ? »*

ENTRÉE : $D = \{(x_i, y_i)\}_{i=1}^n$
SORTIE : Arbre de décision T

FONCTION construire_arbre(D) :

```
— SI condition d'arrêt (profondeur max, nb min d'obs, pureté) :  
    RETOURNER feuille avec prédiction = moyenne( $y$  dans  $D$ )  
  
— POUR CHAQUE feature  $j$  dans  $\{1, \dots, p\}$  :  
    — POUR CHAQUE seuil  $s$  possible :  
        Calculer le gain = Impureté( $D$ ) - [Impureté( $D_{\text{gauche}}$ ) +  
Impureté( $D_{\text{droite}}$ )]  
        (pondéré par la taille des sous-ensembles)  
    — FIN POUR  
— FIN POUR  
  
— Sélectionner le meilleur  $(j^*, s^*) = \text{argmax}(\text{gain})$   
— Splitter  $D$  en  $D_{\text{gauche}} = \{x : x_{j^*} \leq s^*\}$  et  $D_{\text{droite}} = \{x : x_{j^*} > s^*\}$   
  
— Nœud gauche = construire_arbre( $D_{\text{gauche}}$ )  
— Nœud droit  = construire_arbre( $D_{\text{droite}}$ )
```

UNE HEURISTIQUE GLOUTONNE

Le meilleur choix local
à chaque étape, sans retour en
arrière.

Impureté et gain

Impureté = à quel point les données
sont mélangées
Gain = Réduction d'impureté apportée
par un split.

Machine Learning

Construction d'un arbre: exemple assurance auto

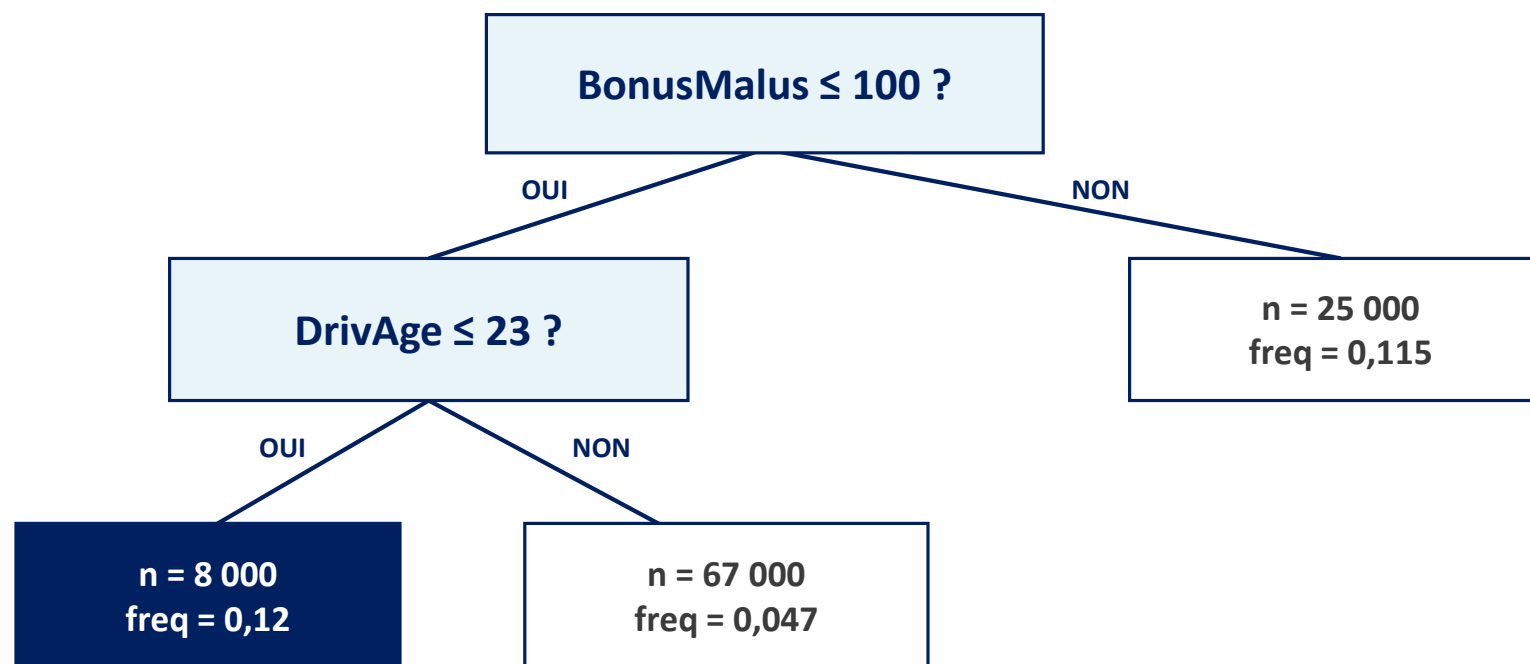
Deux splits suffisent déjà à révéler une interaction entre bonus-malus et âge du conducteur

ÉTAPE 1

100 000 assurés
Fréquence moyenne 0,07
Gain d'impureté 0,0023

ÉTAPE 2

On développe la branche gauche :
meilleur split = $\text{DrivAge} \leq 23$



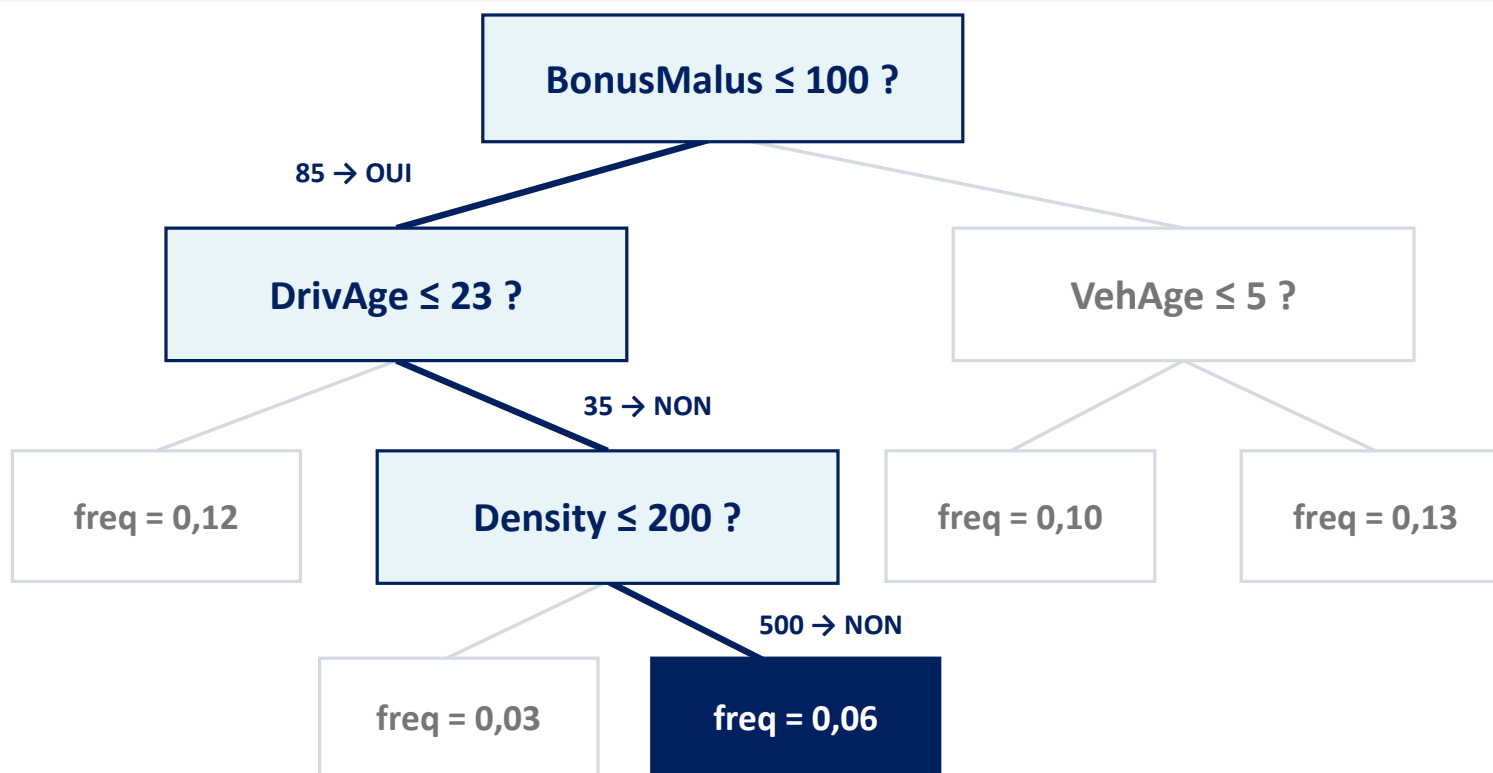
Interaction détectée : même avec un bon bonus, les jeunes conducteurs restent plus risqués.

Machine Learning

Comment un arbre prédit

Une observation descend l'arbre ; la feuille atteinte fournit directement la prédiction

Nouvel assuré · DrivAge = 35 · BonusMalus = 85 · VehAge = 3 · Density = 500



PRÉDICTION

0,06

Chemin entièrement
explicable

Machine Learning

Critères de split : mesurer puis maximiser la baisse d'impureté

Le critère dépend du type de cible ; le gain compare ensuite tous les couples variable–seuil.

CRITÈRE & FORMULE

LECTURE / USAGE

CLASSIFICATION • GINI

$$1 - \sum_k p_k^2$$

Mesure le mélange des classes ; c'est le choix par défaut de scikit-learn.

CLASSIFICATION • ENTROPIE

$$-\sum_k p_k \log_2(p_k)$$

Pénalise davantage le mélange et peut conduire à des partitions plus équilibrées.

RÉGRESSION • MSE

$$\frac{1}{n} \sum_i (y_i - \bar{y})^2$$

Minimise la variance autour de la moyenne ; standard pour une cible continue.

RÉGRESSION • POISSON

$$\frac{1}{n} \sum_i \left(y_i \log \frac{y_i}{\hat{y}} - y_i + \hat{y} \right)$$

Respecte la structure des comptages positifs ; à privilégier pour la fréquence sinistre.

GAIN DU SPLIT

Gain = impureté(parent) – impureté(enfants, pondérée) : CART retient le couple (j*, s*) qui maximise cette baisse.

Machine Learning

Un arbre seul apprend vite , mais peut présenter des instabilités

Quelques observations différentes peuvent changer le premier split. Cela affectera toute la prédiction.

UN SEUL ARBRE · UNE DÉCISION FRAGILE

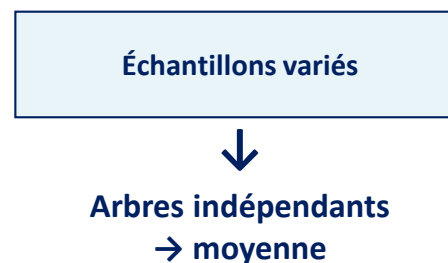


Même problème, données presque identiques :
les splits et la prédiction peuvent basculer.

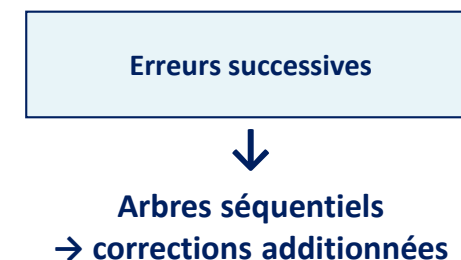
LA RÉPONSE · ENSEMBLE MODELING

**Ne plus faire confiance à un seul arbre :
combiner plusieurs estimateurs imparfaits.**

BOOTSTRAPPING



BOOSTING



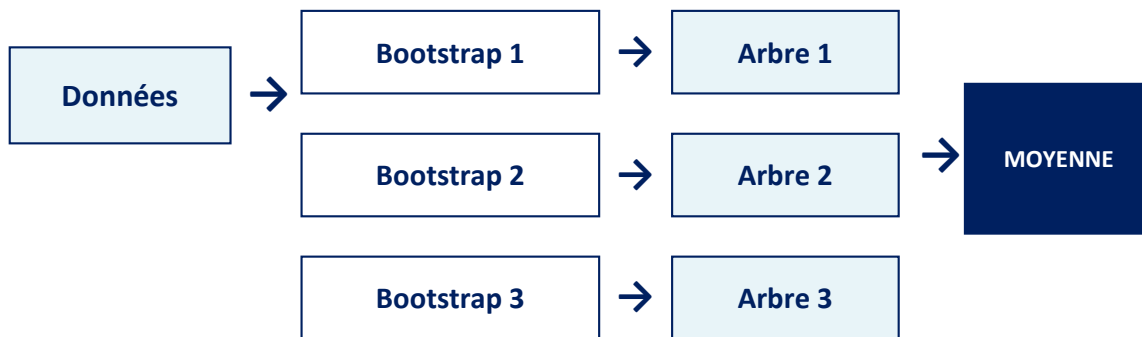
Deux philosophies : stabiliser par la moyenne (bagging) ou apprendre des erreurs (boosting).

Machine Learning

Bagging et boosting corrigent deux faiblesses différentes

À ne pas confondre : le bootstrap est une technique d'échantillonnage ; le bagging et le boosting sont des stratégies d'ensemble.

BAGGING · Bootstrap AGGregatING



... jusqu'à B arbres indépendants

BOOTSTRAP — tirer n observations avec remise dans les n données d'origine ; certaines sont dupliquées, d'autres absentes.

BAGGING — répéter ce tirage, entraîner des arbres indépendants, puis moyenner leurs prédictions.

RANDOM FOREST

BOOSTING



chaque arbre corrige les résidus du précédent



SOMME PONDÉRÉE

Si on pondère chaque arbre par une constante (le learning rate η)
→ XGBoost

Si on pondère chaque arbre par un coefficient qui détermine la performance de l'arbre sur le jeu de validation
→ AdaBoost

BOOSTING — construire les arbres séquentiellement, chacun corrigeant les erreurs laissées par les précédents.

SANS BOOTSTRAP — les corrections utilisent les données, résidus ou erreurs pondérées, sans nouveau tirage aléatoire à chaque étape.

Machine Learning

Moyenner décorrèle ; corriger séquentiellement réduit le biais

Les deux mécanismes se lisent directement dans leurs équations de construction.

BAGGING · variance de la moyenne

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x) \right) = \frac{\sigma^2}{B}$$

Si les B estimateurs étaient indépendants, **la variance serait divisée par B.**

En pratique, les arbres restent corrélés ; la Random Forest tire aléatoirement les variables candidates pour les décorréler davantage.

BOOSTING · correction additive

$$F_T(x) = \sum_{t=1}^T \eta \cdot h_t(x)$$

$$F_{t-1}(x) + \eta \cdot h_t(x) = F_t(x)$$

↑ **petite correction**

h_t apprend ce que le modèle courant ne capture pas encore ; η ralentit chaque pas pour progresser sans sur-corriger.

Random Forest cherche l'indépendance ; le boosting organise au contraire une dépendance utile entre les arbres.

Machine Learning

Le choix oppose robustesse immédiate et performance optimisée

Random Forest constitue une excellente base ; le boosting gagne souvent après un réglage plus fin.

CRITÈRE	BAGGING · RANDOM FOREST	BOOSTING · GBM / XGBOOST
Objectif	Réduire la variance	Réduire le biais
Construction	Parallèle : arbres indépendants	Séquentielle : corrections itératives
Arbres typiques	Profonds, donc peu biaisés	Peu profonds : stumps ou profondeur 3–6
Overfitting	Possible, même si plus facile en boosting	
Tuning	Simple ; peu d'hyperparamètres critiques	Plus fin : learning_rate, n_estimators, depth
Performance	Très bonne et robuste	Souvent la meilleure après réglage
Entraînement	Naturellement parallélisable	Séquentiel, donc généralement plus lent

Réflexe pratique : commencer par Random Forest ; passer au boosting quand le gain de performance justifie le tuning.

Exercice

`ex1.ipynb`

Bloc 2

Les algorithmes d'ensemble

Machine Learning

Bloc 2 — transformer une collection d'arbres en modèle fort

Random Forest, GBM, XGBoost et LightGBM : comprendre leurs mécanismes avant de les régler.

MESSAGE CLÉ

Combiner des modèles faibles pour en faire un modèle fort.

Le bon outil dépend de la façon dont les arbres sont diversifiés, corrigés et régularisés.

2.1	Random Forest	Bagging + décorrélation
2.2	Gradient Boosting	Principe et GBM sklearn
2.3	XGBoost & LightGBM	Implémentations modernes
2.4	Hyperparamètres	Stratégie de tuning
2.5	Comparaison actuarielle	Synthèse
2.6	Exercice pratique	Code

Random Forest

Random Forest décorrèle les arbres avant de les moyenner

POUR CHAQUE ARBRE $b = 1, \dots, B$

1

BOOTSTRAP

Tirer n observations avec remise

Chaque arbre voit un dataset différent.

2

FEATURES ALÉATOIRES

À chaque nœud, tester seulement m variables

Une variable dominante ne s'impose pas partout.

3

ARBRE PROFOND

Faire croître l'arbre T_b sans pruning (régularisation)

Chaque estimateur reste peu biaisé mais instable.

Pour éviter que les B arbres fassent les mêmes splits, on les force à raisonner sur des features différentes avec `max_features`

AGRÉGATION

4

RÉGRESSION

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

ou

CLASSIFICATION
vote majoritaire

Arbres moins corrélés
→ variance réduite

Random Forest

Random Forest : un réglage simple, dominé par max_features

HYPERPARAMÈTRE	VAL. DEFAULT	RÔLE	IMPACT PRATIQUE
n_estimators	100	Nombre d'arbres	500–1 000 : plus stable, mais plus lent
max_features	'sqrt' / 1.0	Variables testées par split	Levier principal de décorrélation
max_depth	None	Profondeur maximale	Limiter rarement ; préférer min_samples_leaf
min_samples_leaf	1	Observations minimales par feuille	Augmenter pour lisser et stabiliser
n_jobs	None	Parallélisation CPU	-1 utilise tous les cœurs

Réflexe : mettez beaucoup d'arbres, puis réglez max_features. Les autres défauts constituent une base solide.

Random Forest

Une configuration robuste tient en quelques lignes de sklearn

Le bootstrap et la pondération d'exposition s'intègrent directement dans l'API standard.

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=500,
    max_features=0.33,
    min_samples_leaf=50,
    n_jobs=-1,
    random_state=42
)
rf.fit(X_train, y_train,
       sample_weight=exposure_train)

importances = pd.Series(
    rf.feature_importances_, index=feature_names
).sort_values(ascending=False)
```

POUR LA TARIFICATION

500 arbres pour stabiliser la moyenne.

max_features = 0.33 pour diversifier les splits.

min_samples_leaf = 50 pour éviter les micro-segments.

sample_weight transmet l'exposition au fit.

`feature_importances_` fournit un
diagnostic.
Ce n'est pas une preuve causale.

Random Forest

Le bootstrap offre une validation interne presque gratuite

Pour chaque arbre, les observations non tirées deviennent son jeu de test Out-of-Bag.

COMMENT SE FORME LE SCORE OOB

Dataset original · n observations

↓ tirage avec remise

≈ 63 % distinctes
ENTRAÎNEMENT de l'arbre

≈ 37 %
OOB

Chaque observation est prédite uniquement par les arbres qui ne l'ont pas vue ; les prédictions sont ensuite agrégées.

EN PRATIQUE

```
rf_oob = RandomForestRegressor(  
    n_estimators=500,  
    oob_score=True,  
    random_state=42  
)  
rf_oob.fit(X_train, y_train)  
  
print(rf_oob.oob_score_) # R2 OOB
```

Une estimation de généralisation proche d'une CV à 5 folds,
sans réentraînements séparés.

Gradient Boosting

Gradient Boosting corrige le modèle, arbre après arbre

Chaque petit arbre apprend ce que la somme actuelle ne prédit pas encore.

INITIALISATION

$$F_0(x) = \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

modèle constant · biais élevé



Une ébauche grossière
à affiner progressivement

À CHAQUE ITÉRATION $t = 1, \dots, T$

1

CALCULER LES RÉSIDUS

$$r_i = y_i - F_{t-1}(x_i)$$

ce que le modèle ne prédit pas encore

2

ENTRAÎNER UN PETIT ARBRE

$$h_t(x)$$

le weak learner prédit les erreurs actuelles

3

AJOUTER UNE CORRECTION

$$F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x)$$

η limite la taille de chaque pas
+ Descente de gradient avec un arbre à chaque étape
de la descente

MODÈLE FINAL

$$F_t(x)$$

$$F_0(x) + \eta h_1(x) + \eta h_2(x) + \dots + \eta h_t(x)$$

↑ couche après
couche

η petit → plus d'arbres, mais une
meilleure régularisation (=shrinkage)

Gradient Boosting

En boosting, chaque arbre doit rester volontairement faible

La profondeur limite les interactions capturées à chaque étape et régularise la somme finale.

RANDOM FOREST

PROFONDS

Chaque arbre est un
« strong learner »

OBJECTIF

Faible biais individuel

COMBINAISON

Moyenne parallèle

EFFET

Variance réduite

GRADIENT BOOSTING

3–6

niveaux par arbre
« weak learners »

OBJECTIF

Petit signal à chaque étape

COMBINAISON

Addition séquentielle

EFFET

Biais global réduit

Profondeur 3 = interactions d'ordre 3 au maximum.

Gradient Boosting

GradientBoostingRegressor rend la trajectoire d'apprentissage observable

```
from sklearn.ensemble import GradientBoostingRegressor

gbm = GradientBoostingRegressor(
    n_estimators=500,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    min_samples_leaf=50,
    random_state=42
)
gbm.fit(X_train, y_train)

scores = []
for y_pred in gbm.staged_predict(X_test):
    scores.append(
        mean_squared_error(y_test, y_pred)
    )
```

LES QUATRE LEVIERS

n_estimators: nombre d'étapes de boosting.

learning_rate: taille de chaque pas.

max_depth: profondeur max de chaque arbre.

Subsample: nb d'échantillon pour fitter chaque arbre.

staged_predict révèle l'itération où la généralisation cesse de s'améliorer.

Pédagogique et fiable, mais lent : XGBoost et LightGBM industrialisent le même principe.

XGBoost & LightGBM

XGBoost et LightGBM industrialisent le Gradient Boosting

Même principe séquentiel, mais calcul, mémoire et régularisation sont profondément optimisés.

ASPECT	SKLEARN GBM	XGBOOST	LIGHTGBM
Vitesse	Lent	Rapide	Très rapide
Mémoire	Élevée	Moyenne	Faible
Catégorielles	Encodage requis	Encodage requis	Natif
Valeurs NaN	Imputation	Natif	Natif
GPU	Non	Oui	Oui
Régularisation	Basique	L1 / L2	L1 / L2
Early stopping	Basique	Intégré	Intégré
API sklearn	Native	Compatible	Compatible

XGBoost & LightGBM

XGBoost régularise la structure et exploite la courbure de la perte

Deux innovations expliquent sa robustesse : pénaliser l'arbre et utiliser une approximation de Newton.

1 • RÉGULARISATION DANS L'OBJECTIF

$$\text{Obj} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{t=1}^T \underbrace{\left[\gamma \cdot T_t + \frac{1}{2} \lambda \sum_{j=1}^{T_t} w_j^2 \right]}_{\text{Régularisation de la complexité}}$$

$\gamma \cdot T_t$ pénalise le nombre de feuilles.

$\lambda \sum_{j=1}^{T_t} w_j^2$ limite la magnitude des prédictions dans les feuilles.

Résultat : **une régularisation structurelle directe.**

2 • GRADIENT + HESSIENNE

$$w_j^* = - \frac{\sum_{i \in \text{feuille } j} g_i}{\sum_{i \in \text{feuille } j} h_i + \lambda}$$

$g_i = \frac{\partial l}{\partial \hat{y}_i}$ mesure la pente de la perte.

$h_i = \frac{\partial^2 l}{\partial \hat{y}_i^2}$ mesure sa courbure.

Résultat : **des pas plus précis et une convergence plus rapide.**

Toute perte deux fois dérivable devient possible: notamment la déviance de Poisson.

Tuning du Gradient Boosting

Quatre leviers expliquent l'essentiel des gains

Capacité, stochasticité et crédibilité statistique : le reste affine surtout les marges.

01	<code>n_estimators × learning_rate</code>	Plus d'arbres + pas plus faible Early stopping choisit le bon nombre	LR ↓ • arbres ↑
02	<code>max_depth / num_leaves</code>	Contrôle la complexité et les interactions capturées par chaque arbre	XGB : 3–6 LGBM : 15–63
03	<code>subsample + colsample_bytree</code>	Introduit de l'aléa et réduit la corrélation entre arbres successifs	Plage utile : 0,7–0,9
04	<code>min_child_weight / min_samples_leaf</code>	Impose une taille minimale aux feuilles et stabilise les prédictions	Assurance : 50–100 obs. par feuille

Ordre pratique : structure de l'arbre → stochasticité → learning rate final.

Tuning du Gradient Boosting

Une procédure en trois étapes évite la recherche exhaustive

Fixer une trajectoire d'apprentissage, régler la structure, puis affiner la régularisation.

01 CALIBRER LA TRAJECTOIRE

`learning_rate = 0,1`
`n_estimators` élevé
+ early stopping

Sortie : nombre d'arbres optimal

02 RÉGLER LA STRUCTURE

`RandomizedSearchCV`
`max_depth` $\in \{3,4,5,6,7\}$
`min_child` $\in \{10,30,50,100\}$

Sortie : capacité de chaque arbre

03 AFFINER ET FINALISER

`subsample, colsample`
 $\in \{0,7 ; 0,8 ; 0,9 ; 1,0\}$
Puis `LR = 0,01-0,05`

Sortie : modèle final régularisé

En général, l'étape 1 avec les paramètres par défaut et l'early stopping produit souvent déjà un excellent modèle.

Choix de l'algorithme

Le meilleur modèle dépend d'abord de votre contrainte dominante

Parcourir les questions dans l'ordre permet de réduire rapidement l'espace des options.

01	Interprétabilité maximale ?	GLM ou arbre peu profond	coefficients ou règles lisibles
02	Peu de données : $n < 5\,000$?	Random Forest	robuste, peu de tuning
03	Tabulaire + catégories ou NaN ?	LightGBM	rapide, gestion native
04	Prototype avec tuning minimal ?	Random Forest ou LightGBM	bons défauts de départ
05	Performance maximale ?	XGBoost ou LightGBM	tuning + early stopping

Hors données tabulaires : texte → transformers / images → CNN / modèles de vision.

Exercice

`ex2.ipynb`